AN ILLUSTRATED GUIDE TO YOUR APP

Recipe Saver — how it works

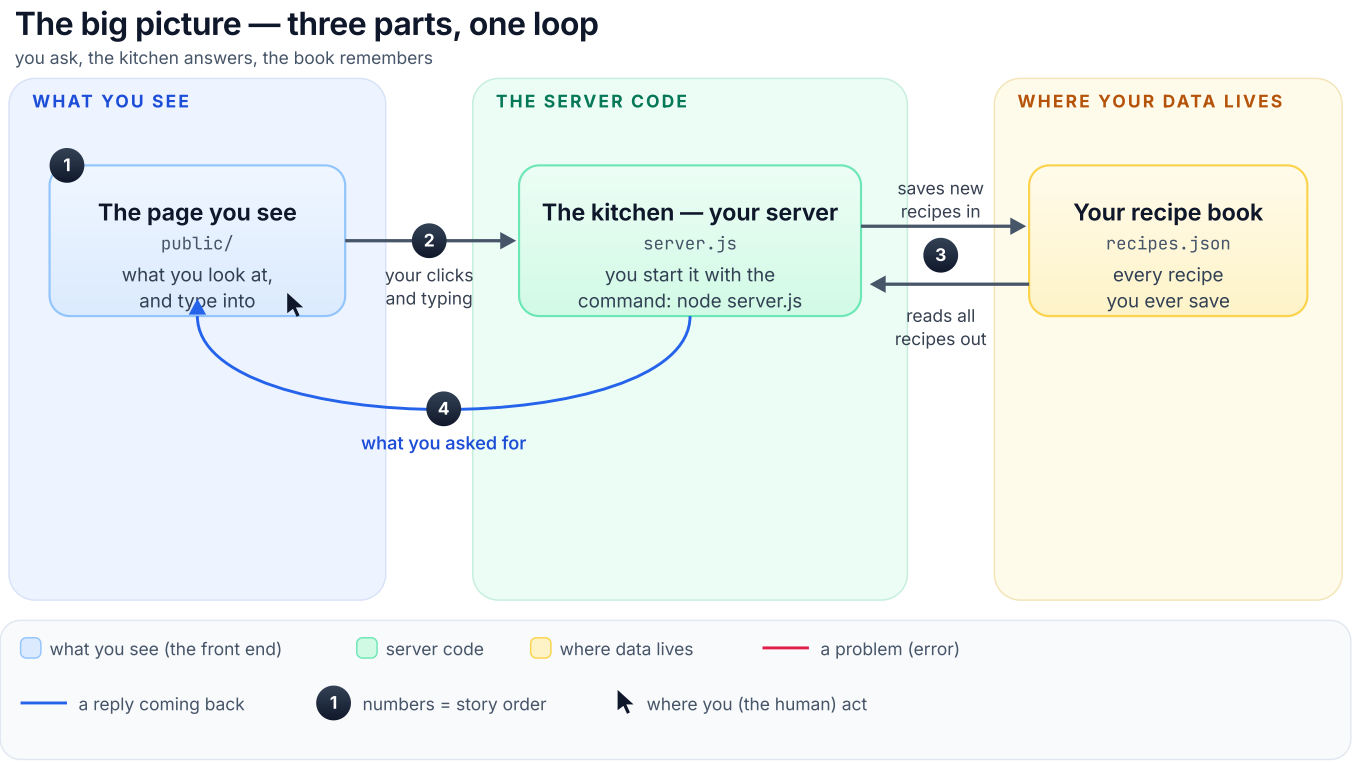
IN PLAIN WORDS

Recipe Saver is a small recipe book that lives entirely on your computer. The page you see in your **browser** (the program you are reading this in — Chrome, Safari, Firefox) is the table: a form, a list, a search box. Behind it, a small program called the **server** (the program that listens for requests and answers them) is the kitchen: it checks every recipe you save and keeps the book. The book itself is one file, **recipes.json**. Every click you make travels from the table to the kitchen and an answer travels back — and that loop is the whole app.

01 · THE MAP

The big picture

Three parts, one loop. Once you can read this picture, every other one below is a zoom-in.



In plain words: 1 The page is your table — you look, click, and type. 2 Everything you do is carried to the kitchen: your server, **server.js**, which you start with the command **node server.js**. 3 The kitchen writes new recipes into your book (**recipes.json**) and reads all recipes back out of it. 4 The answer comes back to your page. It is a restaurant at your own desk: menu in front of you, kitchen behind the wall, recipe book in the kitchen. Every click is one loop of ordering and serving — hold this picture and the rest of this document is just zoom.

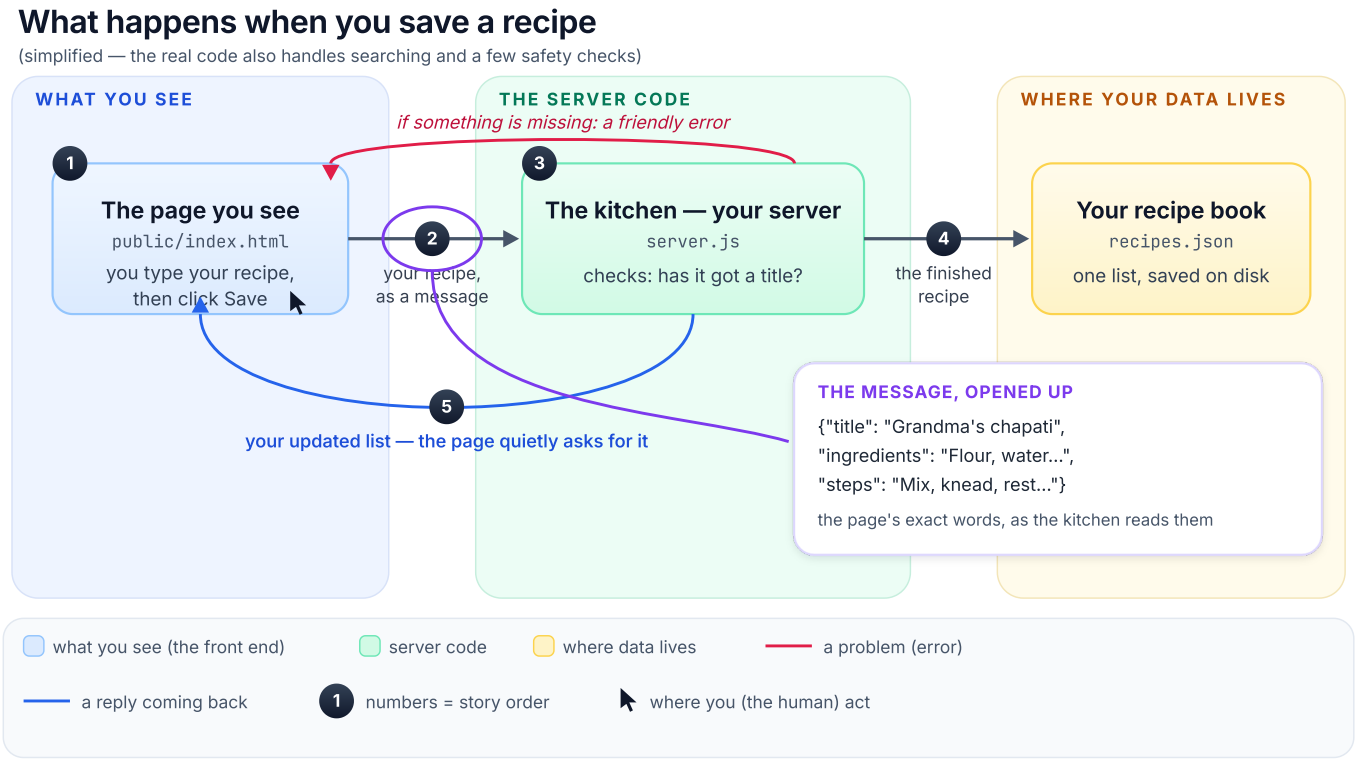
02 · THE STORIES

What happens when you…

Two things you can do, two little trips behind the screen. Both start with you, and both end with your page quietly redrawing itself.

…save a recipe

You fill in the title, maybe the ingredients and steps, and click *Save recipe*. Behind the scenes, five things happen in about a blink:



In plain words: 1 You type your recipe and click *Save*. 2 The page wraps what you typed into a message — the violet card shows exactly what it looks like on its way over. 3 The kitchen does one quality check — does it have a title? If something is missing, the red arrow sends a friendly error straight back to your form and nothing gets saved. 4 If the recipe passes, the kitchen writes it on a fresh page of your book, right next to everything you saved before. 5 Then, quietly and without you doing anything, your page asks for the whole book again and redraws the list — that is why your new recipe appears instantly.

...search for a recipe

Every letter you type in the search box triggers the same little trip. Here it is with the word *chap*:



In plain words: 1 You type *chap* in the search box. 2 The page sends your search words to the kitchen as a question: *any recipes with "chap" in the name or the ingredients?* 3 The kitchen pulls out the whole book and flips through every page, keeping only the matches. 4 Only the matching recipes come back, and your list redraws with just those. The search is not magic — the kitchen reads your entire book every single time and simply skips what does not match. That is why it feels instant with ten recipes, and why a million-recipe book would need a smarter kitchen.

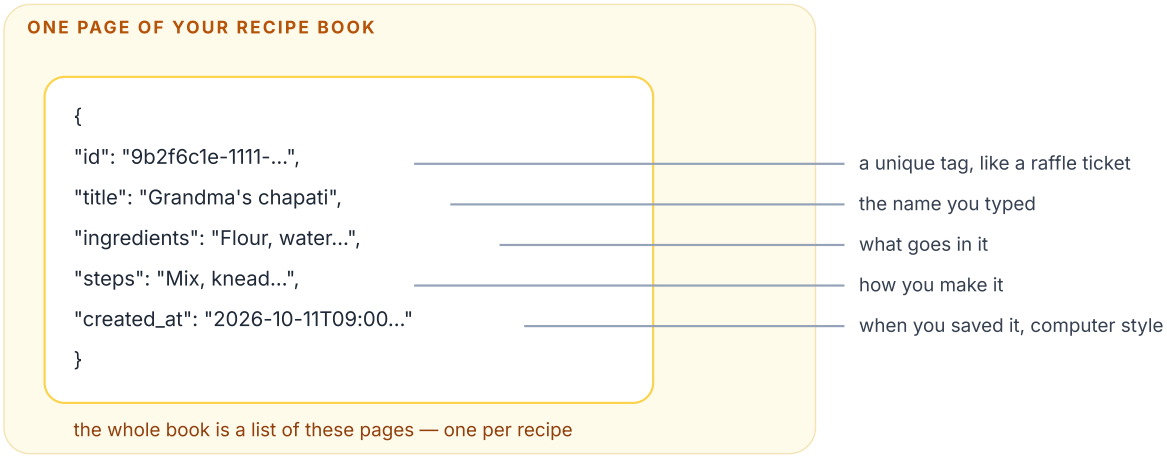
03 · THE DATA

Where your recipes live

Your book is a plain text file, **recipes.json**, sitting next to the code. You can open it in any text editor — here is one page of it, opened up and translated:

Where your recipes live — inside one saved recipe

the file recipes.json, opened up — this is real, you can open it in any text editor



In plain words: one saved recipe is a little form with five boxes. You filled in three — the name, the ingredients, the steps, exactly as you typed them. The kitchen stamped the other two for free: a unique tag, so no two pages in the book can ever get mixed up, and the exact moment you saved it. The curly braces and quotation marks are just the book's grammar — a way of writing forms in plain text that both the page and the server can read (the internet's dialect for this is called **JSON**, and yes, people pronounce it "Jason").

04 · THE FILES

The cast of characters

Every file in your project, one human sentence each — including what would happen if it went missing:

```
recipe-saver/
├── server.js        → the kitchen — starts with: node server.js. Delete it and nothing
│                      the page never loads.
├── public/          → the table and menu — everything your browser shows you
│   ├── index.html   → the page itself: the form, the search box, the list. Delete it an
│   │                  browser shows an error instead of a page.
│   ├── app.js       → the waiter — carries your clicks to the kitchen and brings answer
│   │                  Delete it and the page loads, but nothing happens when you click.
```

```
│   └── style.css     → the decor — colors, fonts, spacing. Delete it and everything stil
│                       works, just ugly.
├── recipes.json      → the recipe book — every recipe you ever saved, one page per recip
│                       Delete it and the app still runs, with an empty book. This file i
│                       only place your recipes exist — copy it somewhere safe if you lov
├── package.json      → the label on the box — the app's name and its one start command.
└── README.md         → the note you (or your AI) left for future you.
```

05 · THE WORDS

Words you'll hear

Every technical word this document used, in the order you met it:

browser
The program you are reading this in — Chrome, Safari, Firefox. It opens pages and talks to servers for you.

server
The program that listens for requests and answers them. Yours is **server.js**, and it runs on your own computer, not on the internet.

page / front end
Everything you can see and click: the form, the list, the search box. The files in **public/**. "Front end" is simply the tech name for "the part you see".

request / response
One trip: a question your page sends to the server, and the answer that comes back. Every click silently sends one.

API
The kitchen's service window: the fixed menu of messages the server understands. Your kitchen understands three — *show me the recipes*, *save this recipe*, *search for this word* — written as the **/api/...** addresses you may spot in the code.

JSON

"Jason" — the plain-text grammar for writing little labeled forms, the one you saw inside a saved recipe. The whole book, and every message between page and kitchen, is written in it.

localhost / port

Your own computer's nickname in the browser. **localhost:3000** means "this same machine, door number 3000" — the port is just a numbered door, and the kitchen listens at door 3000.

validation

The kitchen's quality check before cooking: "does this recipe have a title?" The polite red error you saw is validation talking.

unique id

A made-up tag, long and random, that no two recipes share — like a raffle ticket stapled to each page of the book.

400 and 404

The kitchen's numbered ways of saying no. 404: "never heard of that dish" (wrong address). 400: "this order doesn't make sense" (something was missing or unreadable).

06 · THE SHORT VERSION

How to explain it to a friend

> *"Recipe Saver is a digital recipe book that lives on my computer. A page in my browser is the table, a small program called the server is the kitchen, and a single file is the book. When I save a recipe, the kitchen checks it, glues it into the book, and reads the whole book back to my page. When I search, the kitchen flips through every page and serves up only the matches. Nothing goes online — it's all mine."*

Explained with Explain It With Pictures — a free, open-source skill for AI coding agents.